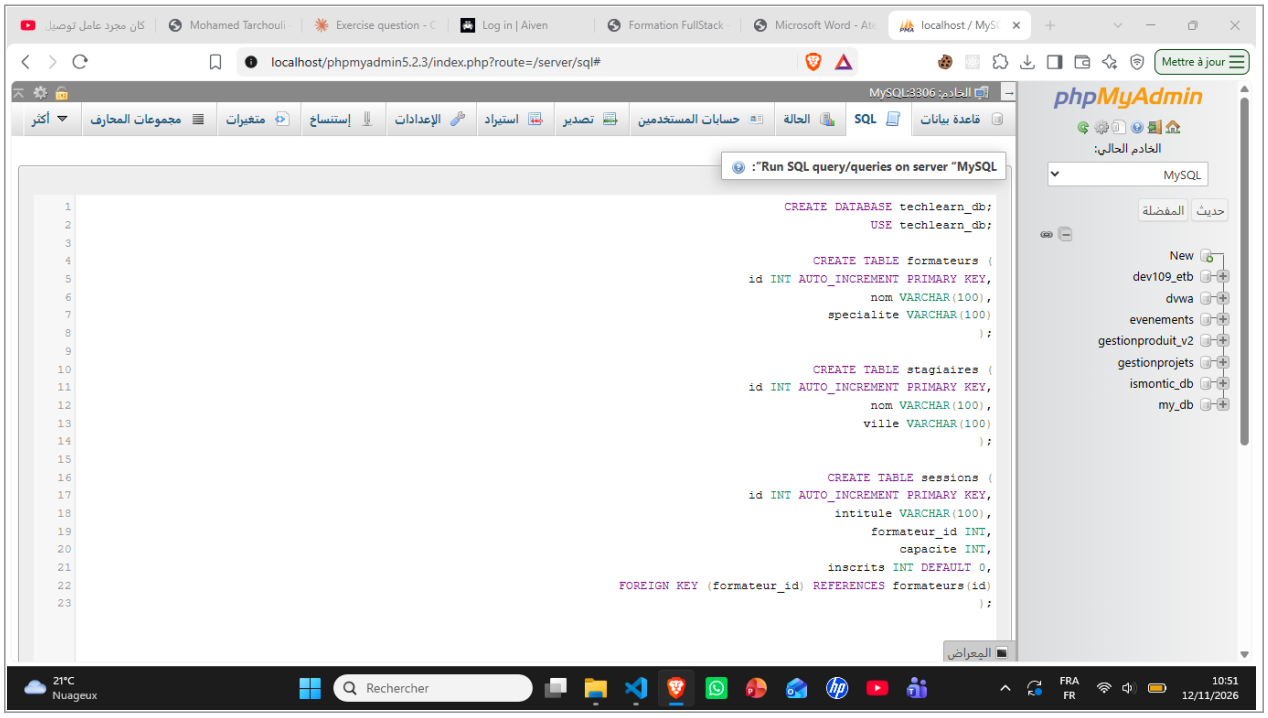
Atelier 3 – Gestion d’un centre de formation

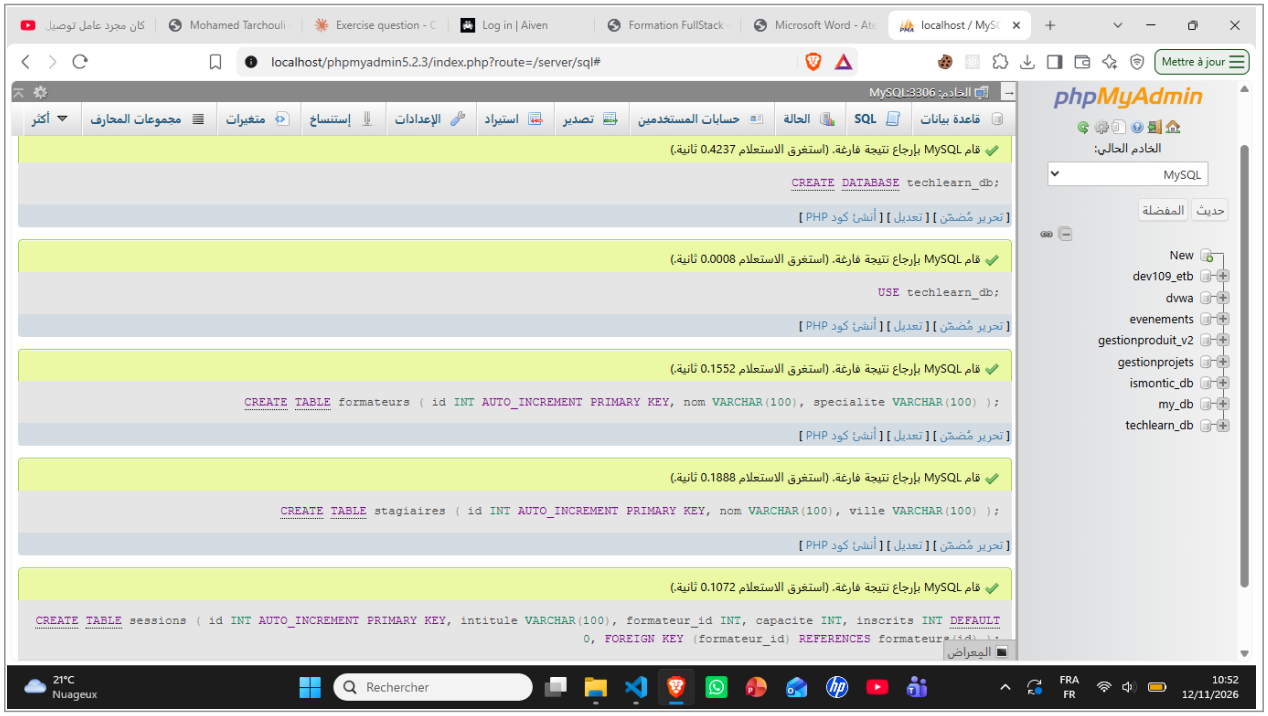
Procédures et fonctions stockées MySQL (base techlearn_db, phpMyAdmin)

1. Création de la base et des tables

Capture 1. Saisie du script dans l’onglet SQL : création de la base techlearn_db et des trois tables formateurs, stagiaires et sessions (la clé étrangère formateur_id relie une session à son formateur).

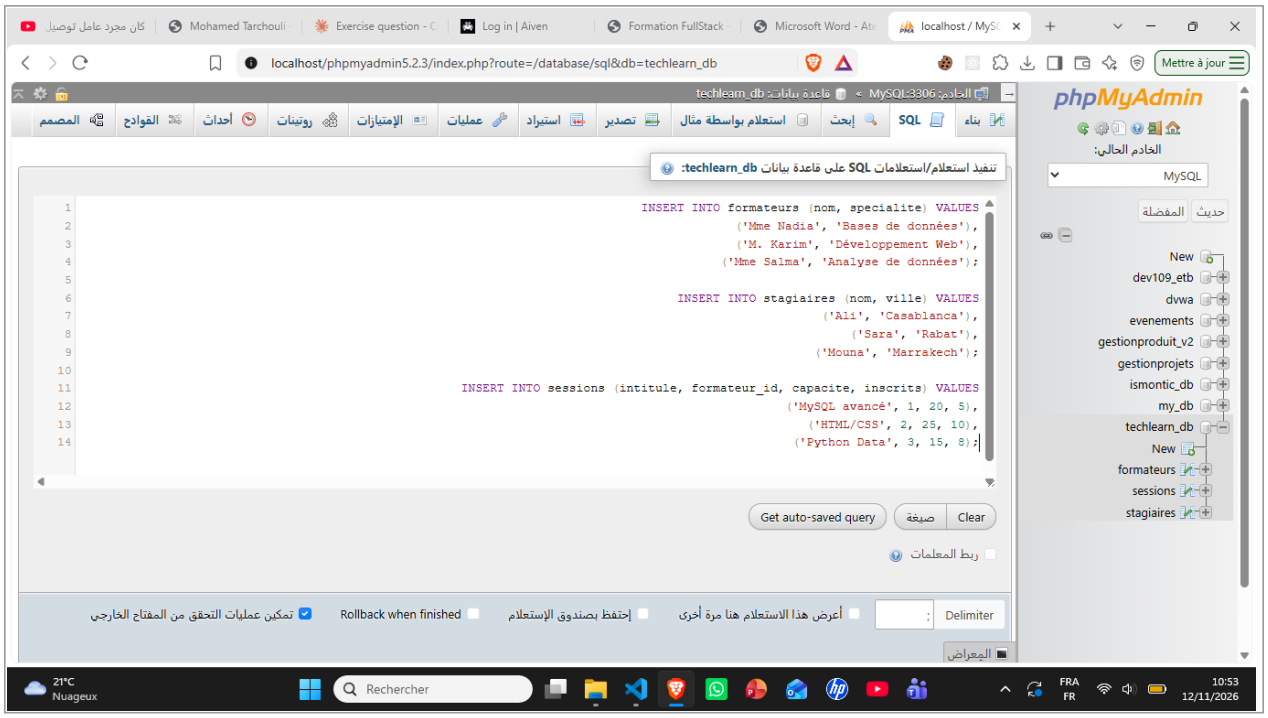


Capture 2. Exécution réussie : la base et les trois tables sont créées sans erreur.

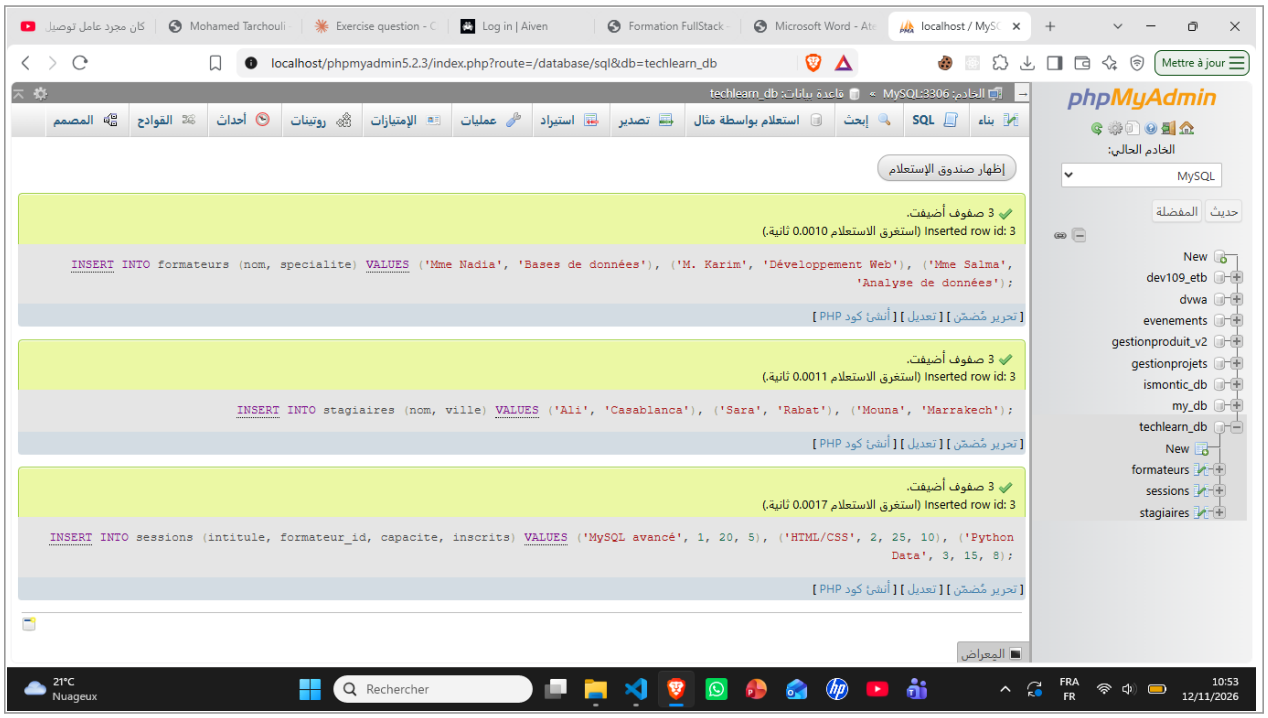


2. Insertion des données

Capture 3. Script d’insertion : 3 formateurs, 3 stagiaires et 3 sessions.

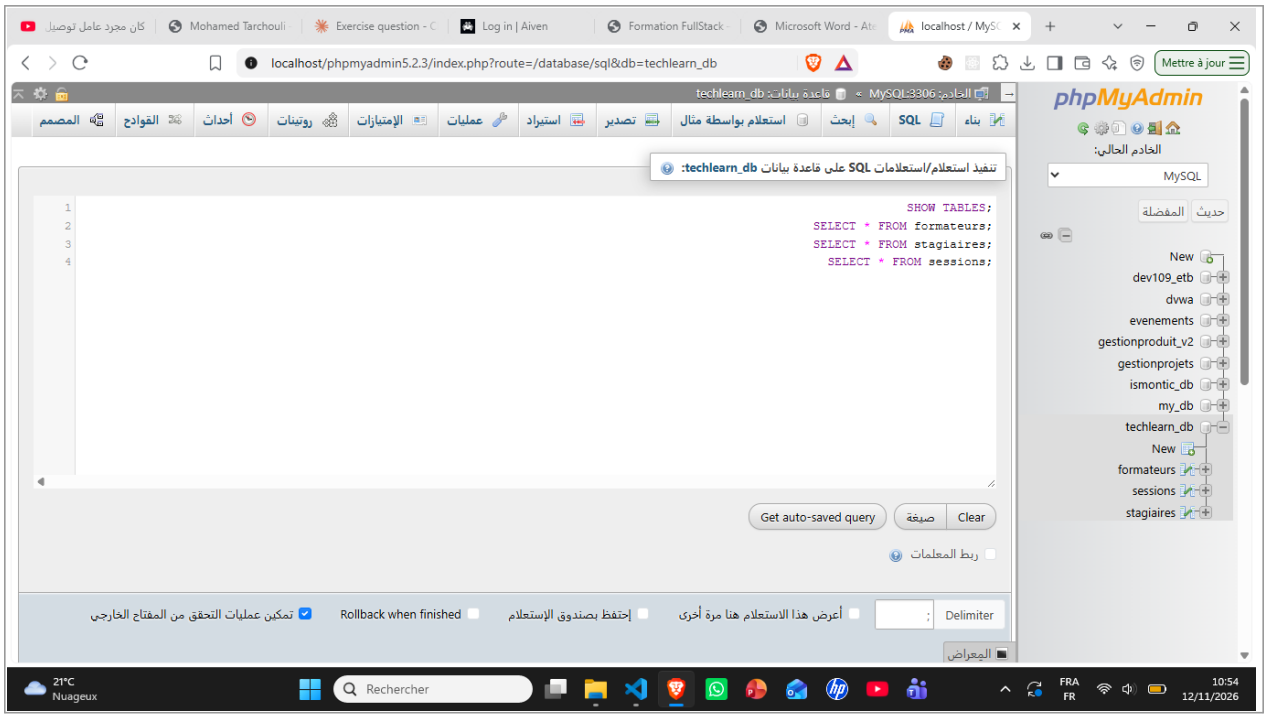


Capture 4. Les trois requêtes INSERT ont réussi : 3 lignes ajoutées dans chaque table.

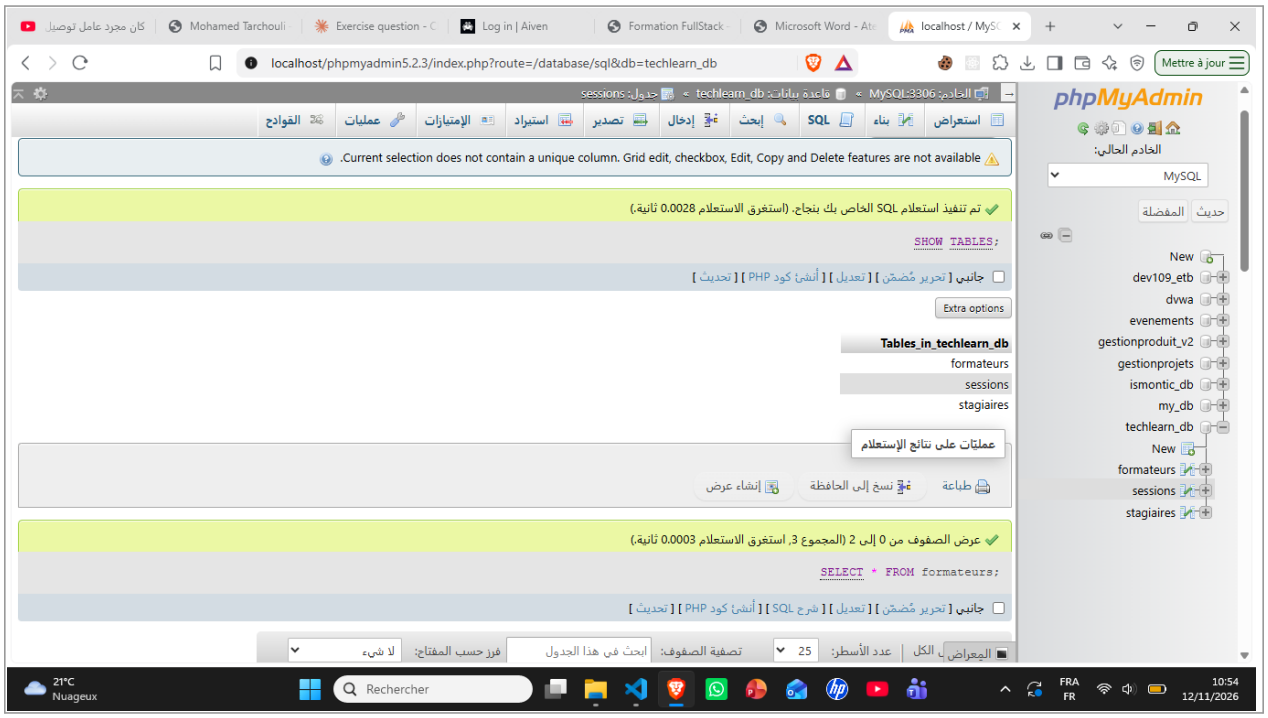


3. Vérification du contenu

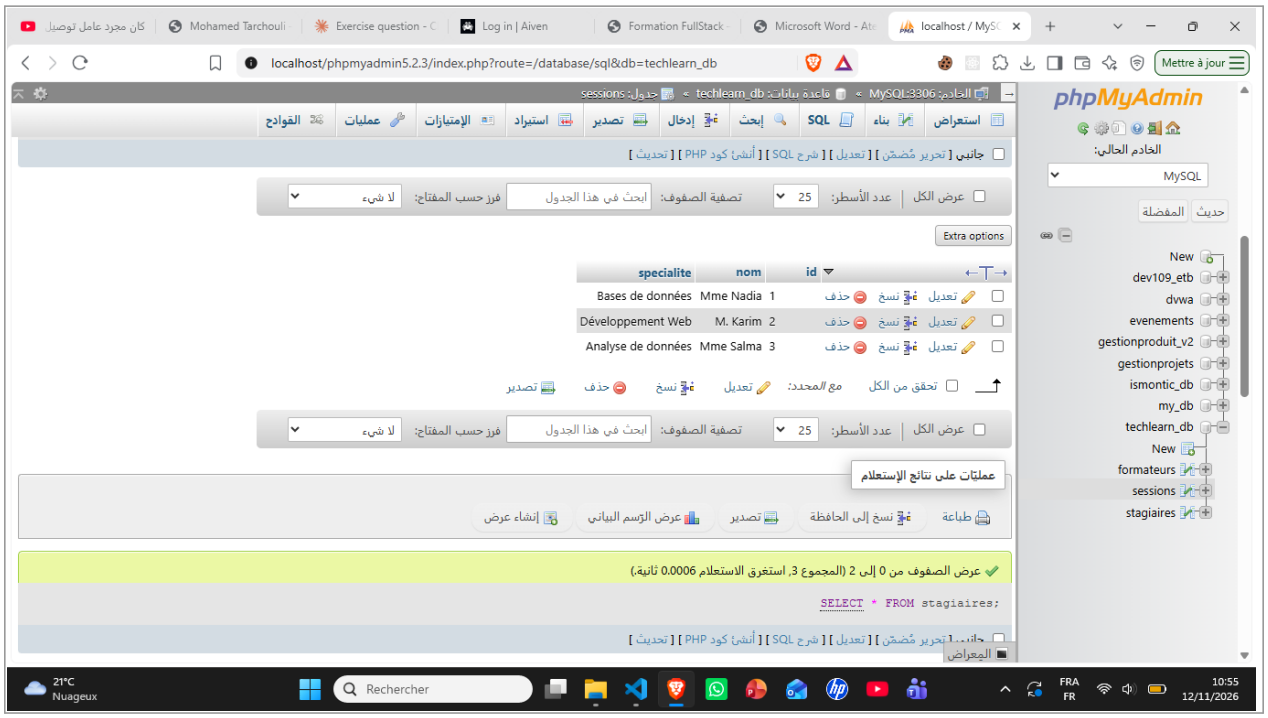
Capture 5. Requêtes de vérification : SHOW TABLES puis SELECT * sur les trois tables.



Capture 6. SHOW TABLES affiche bien les tables formateurs, sessions et stagiaires.



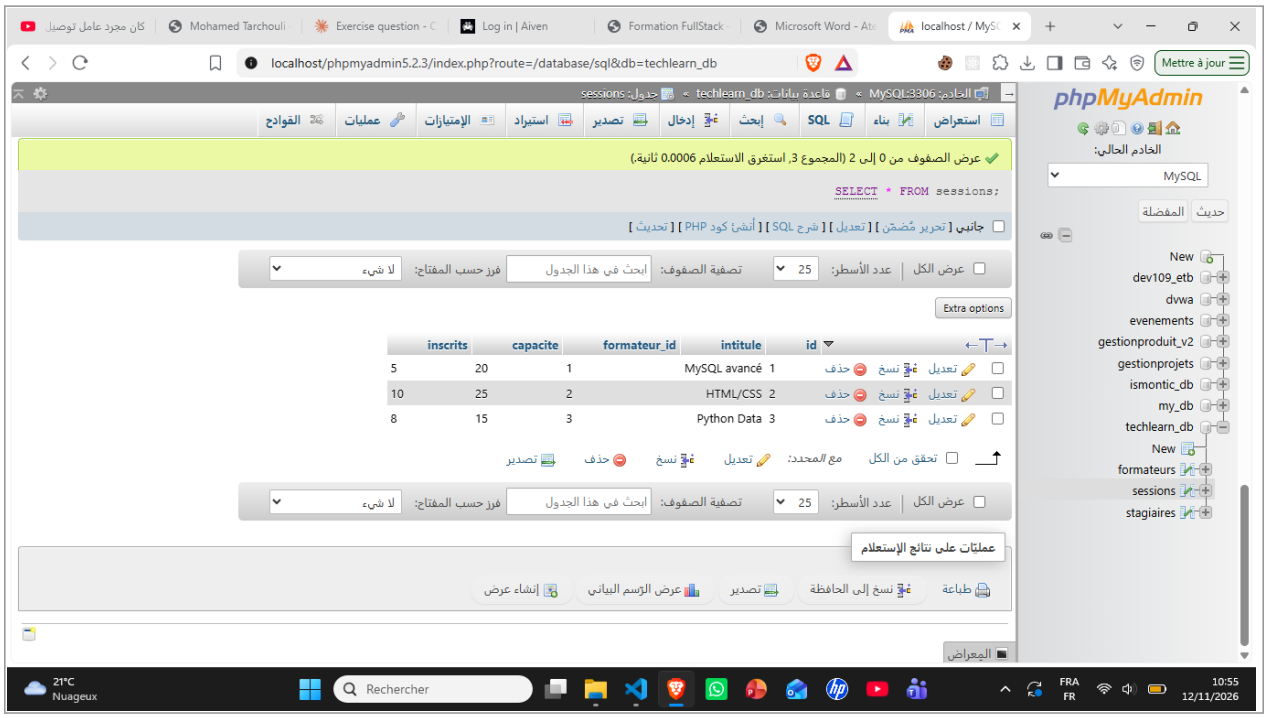
Capture 7. Table formateurs : Mme Nadia (Bases de données), M. Karim (Développement Web), Mme Salma (Analyse de données).



Capture 8. Table stagiaires : Ali (Casablanca), Sara (Rabat), Mouna (Marrakech).

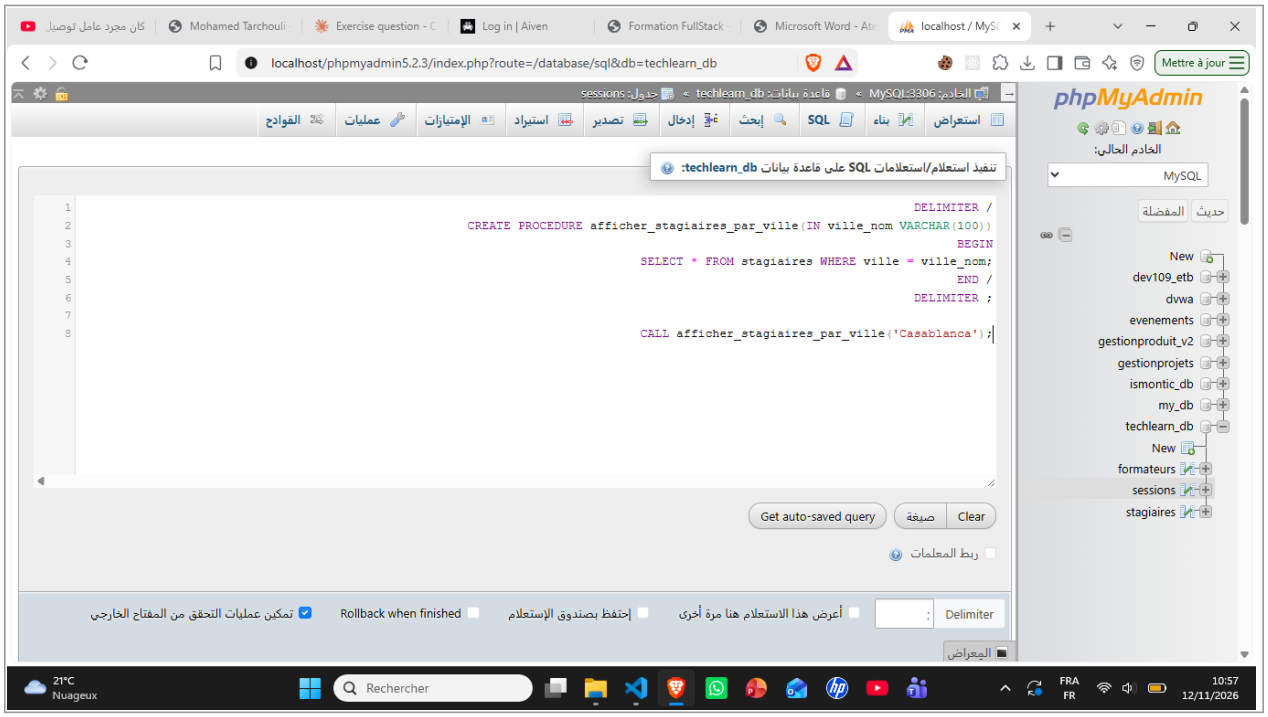


Capture 9. Table sessions avant les exercices : MySQL avancé (5/20), HTML/CSS (10/25), Python Data (8/15).

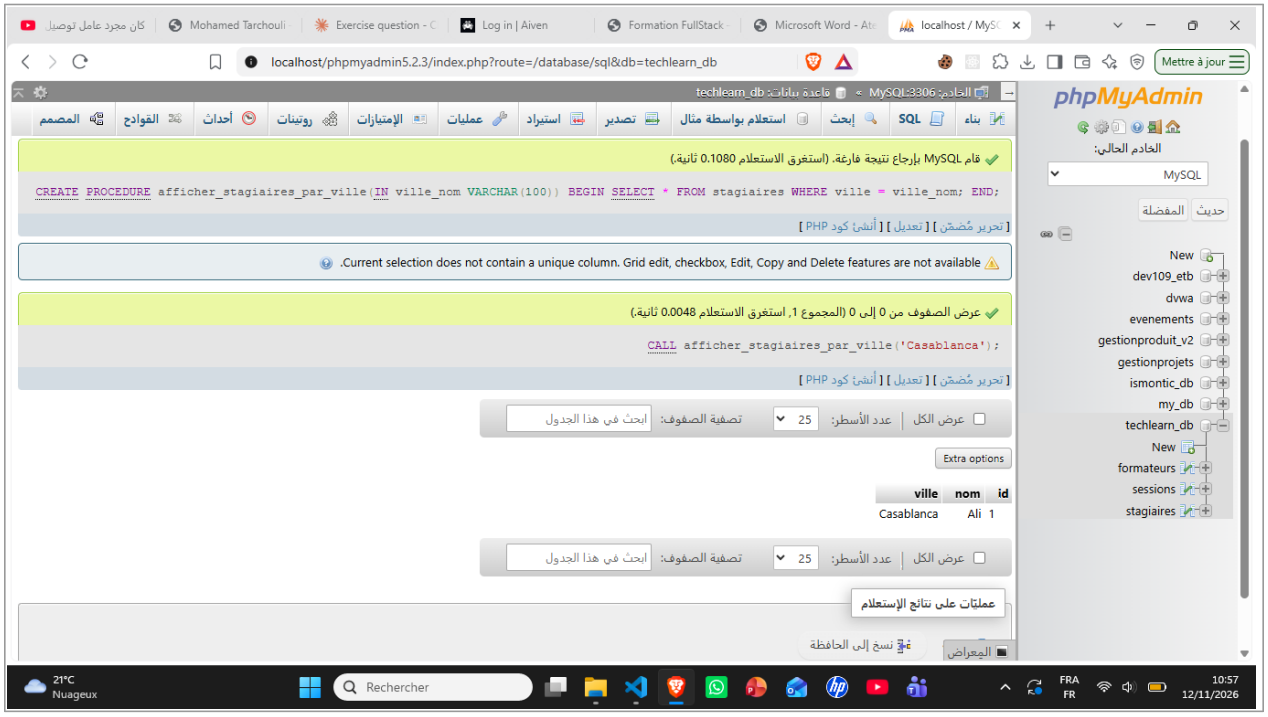


4. Exercice 1 – Procédure avec paramètre IN

Capture 10. Création de la procédure afficher_stagiaires_par_ville. Le paramètre IN ville_nom sert de filtre dans la requête SELECT. Appel avec la valeur « Casablanca ».

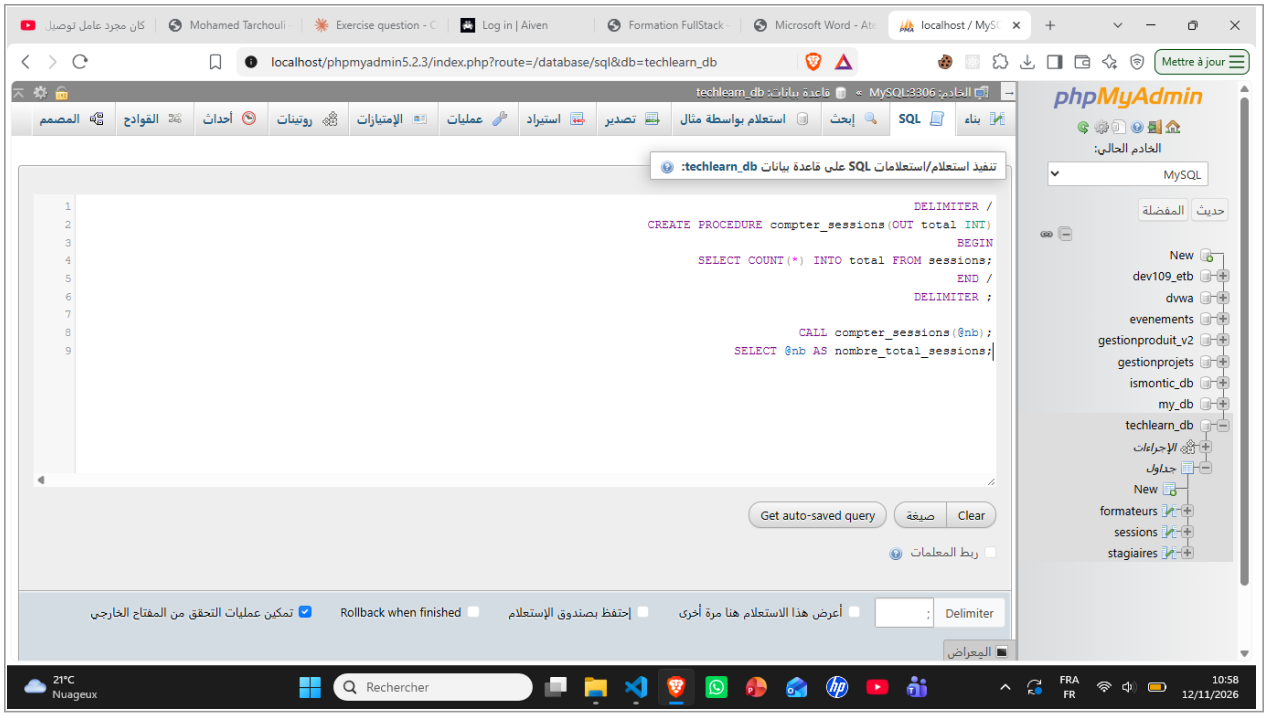


Capture 11. Résultat : seul le stagiaire Ali est affiché, car il est le seul à habiter Casablanca.

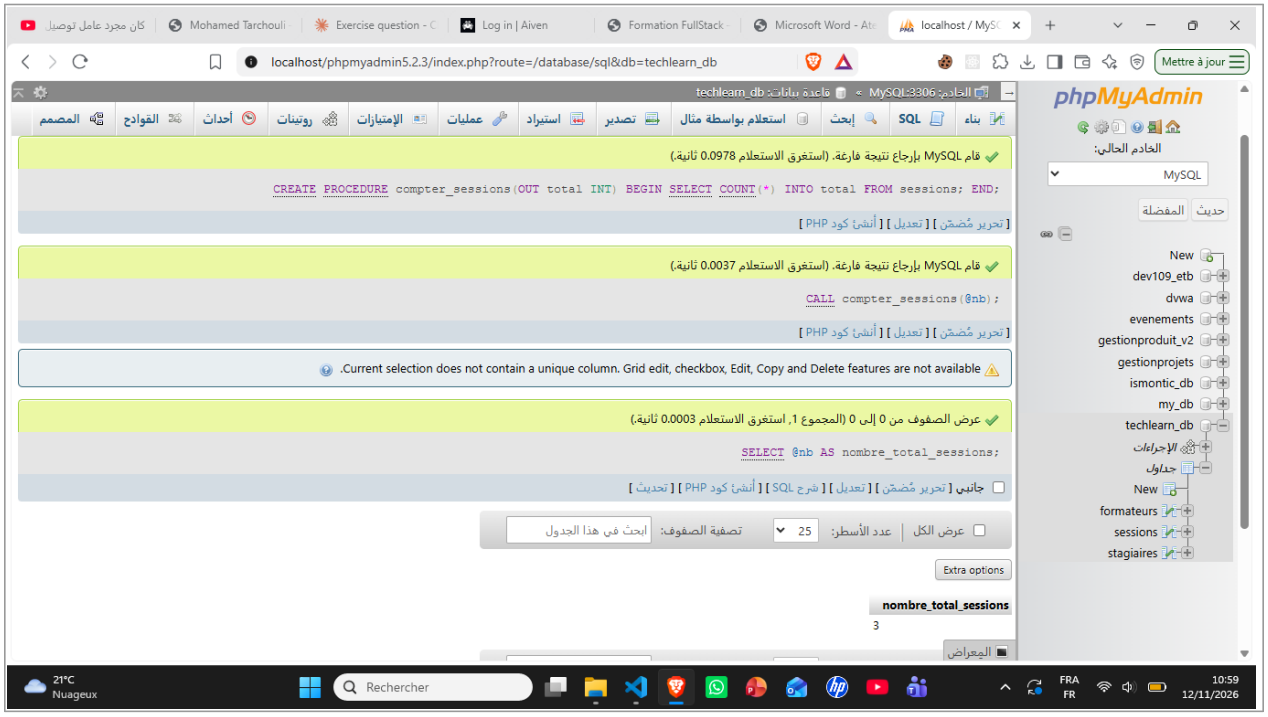


5. Exercice 2 – Procédure avec paramètre OUT

Capture 12. Création de la procédure compter_sessions. Le paramètre OUT total reçoit le résultat de COUNT(*). Appel avec la variable @nb, puis lecture de cette variable.

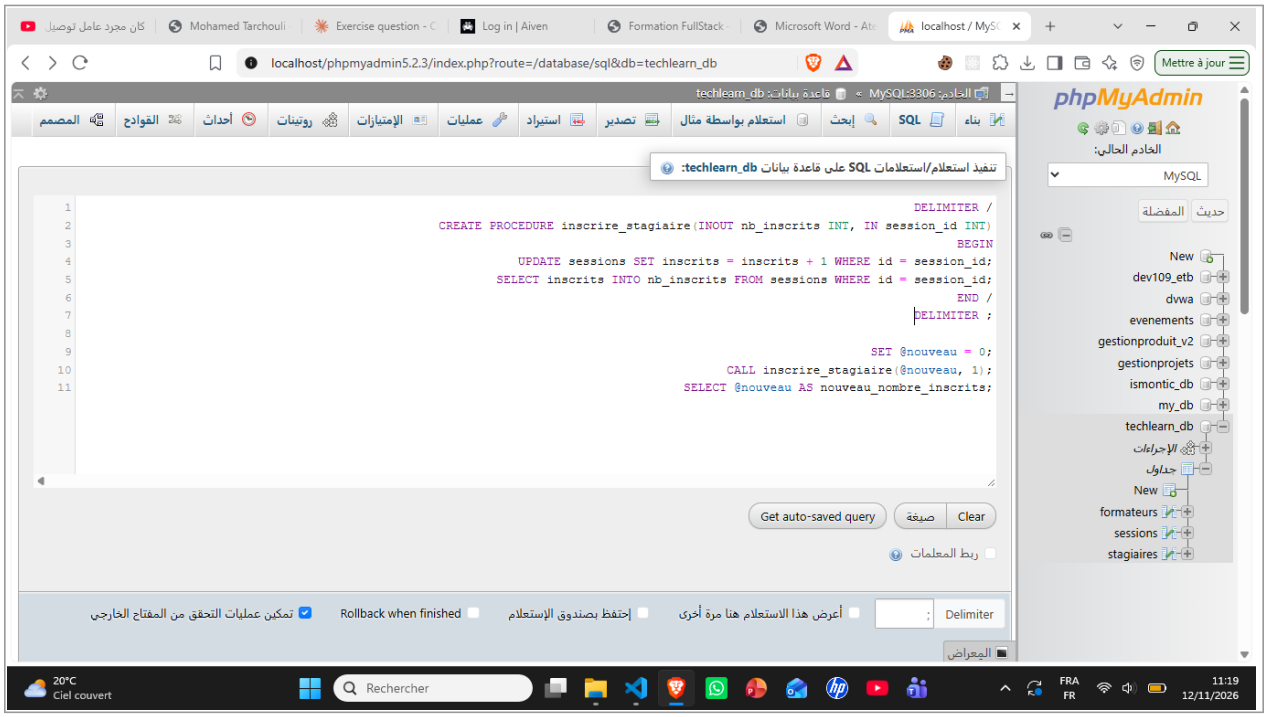


Capture 13. Résultat : nombre_total_sessions = 3, ce qui correspond aux trois sessions insérées.

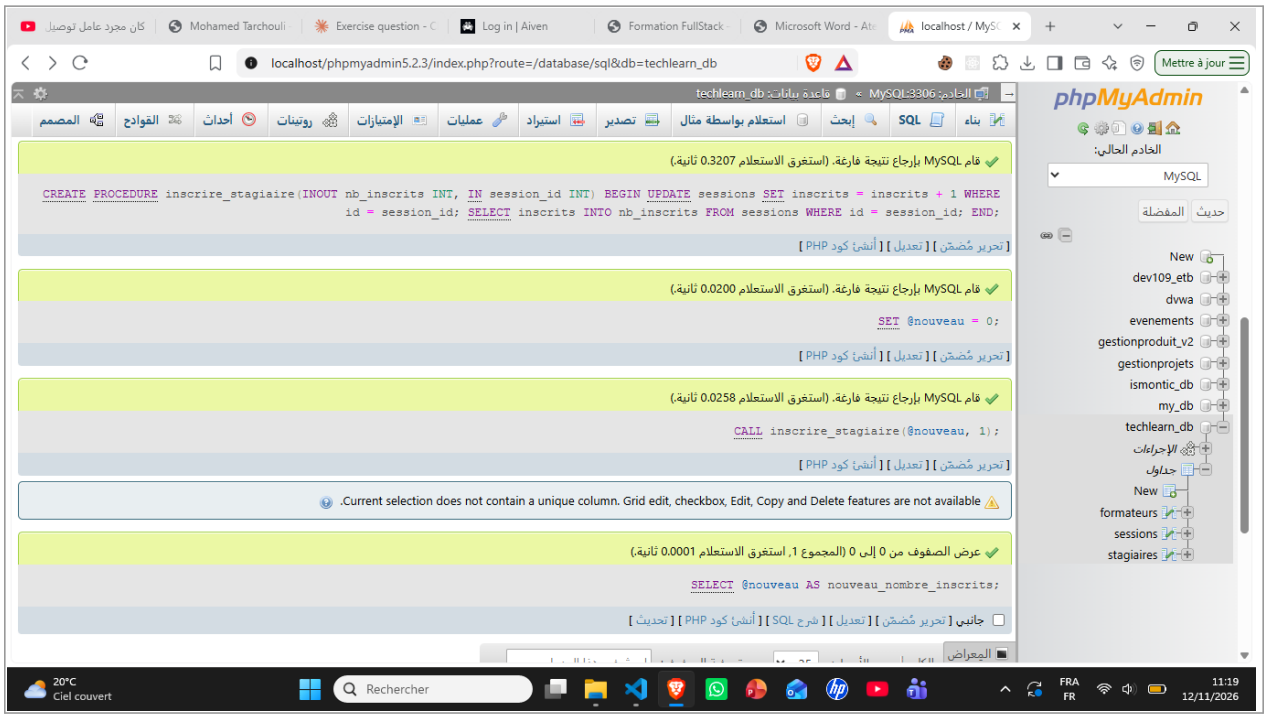


6. Exercice 3 – Procédure avec paramètre INOUT

Capture 14. Création de la procédure inscrire_stagiaire. Elle ajoute 1 au nombre d’inscrits de la session choisie (paramètre IN) et renvoie le nouveau total dans le paramètre INOUT nb_inscrits. La variable @nouveau est initialisée à 0 avant l’appel sur la session 1.

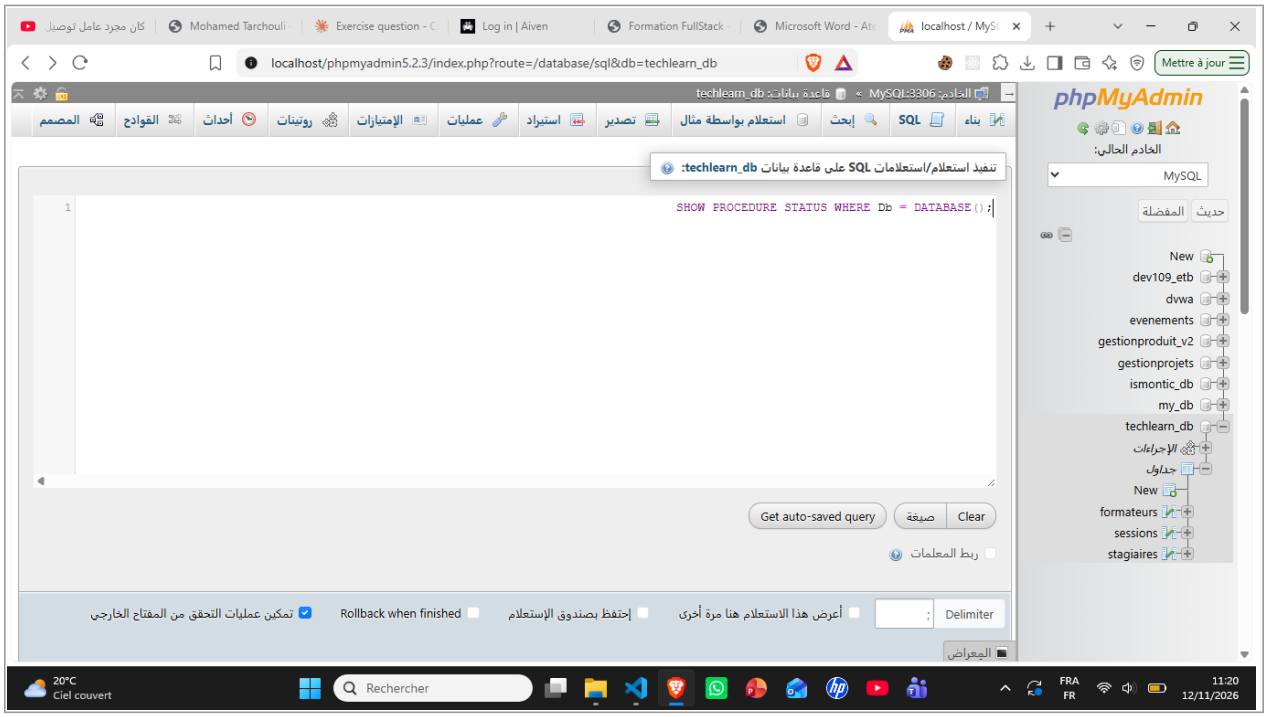


Capture 15. Exécution : la procédure est créée, l’appel réussit et la variable @nouveau est lue. La session 1 passe de 5 à 6 inscrits (visible dans la table sessions aux captures suivantes).

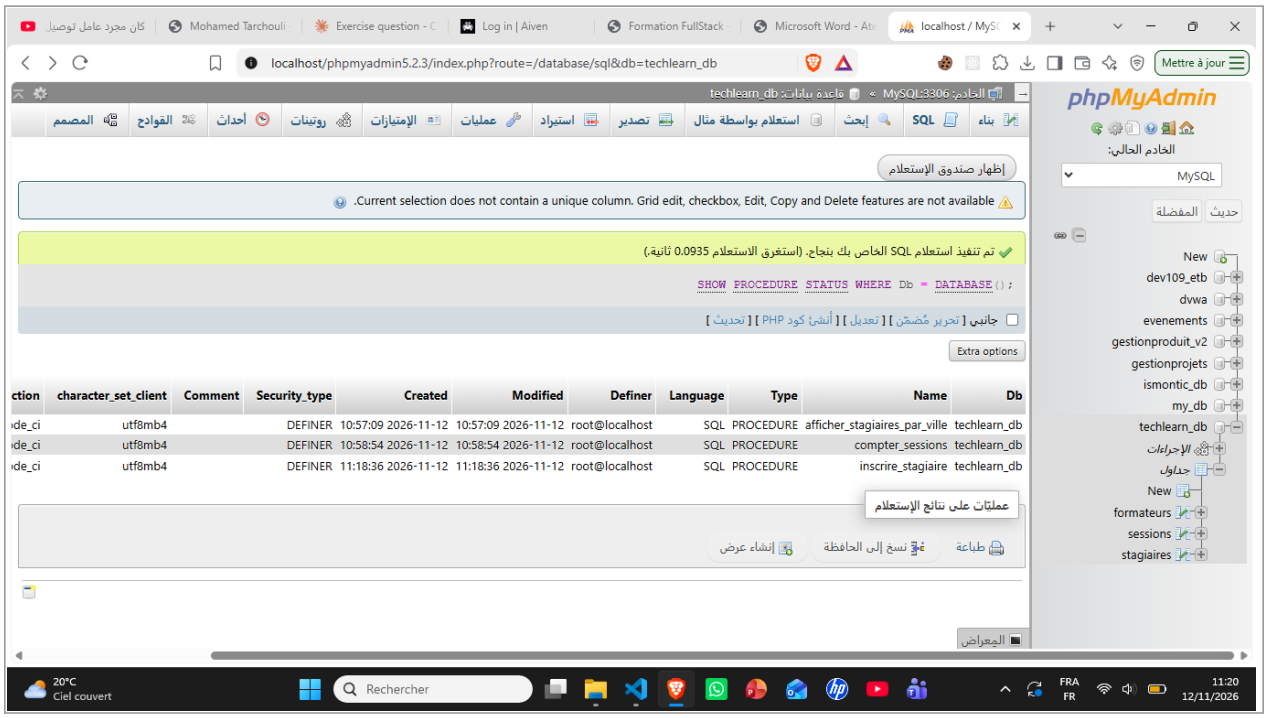


7. Liste des procédures créées

Capture 16. Commande SHOW PROCEDURE STATUS WHERE Db = DATABASE() pour lister les procédures de la base.

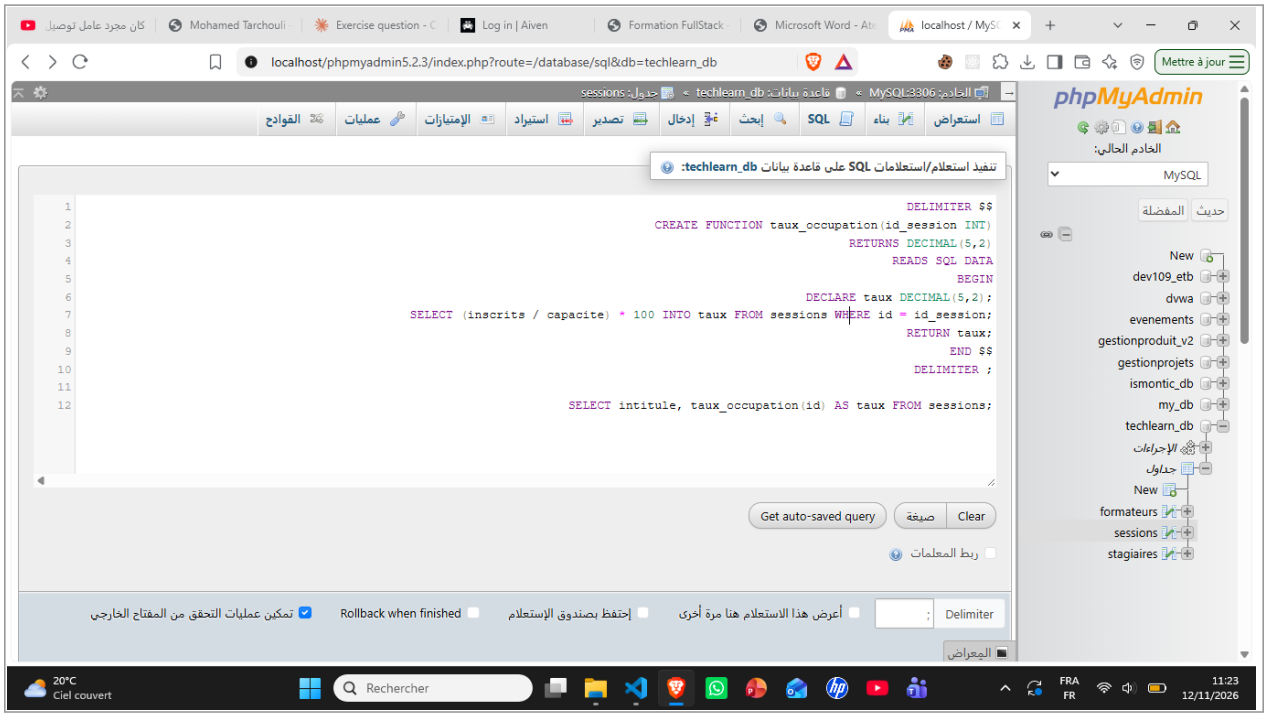


Capture 17. Les trois procédures créées apparaissent : afficher_stagiaires_par_ville, compter_sessions et inscrire_stagiaire.

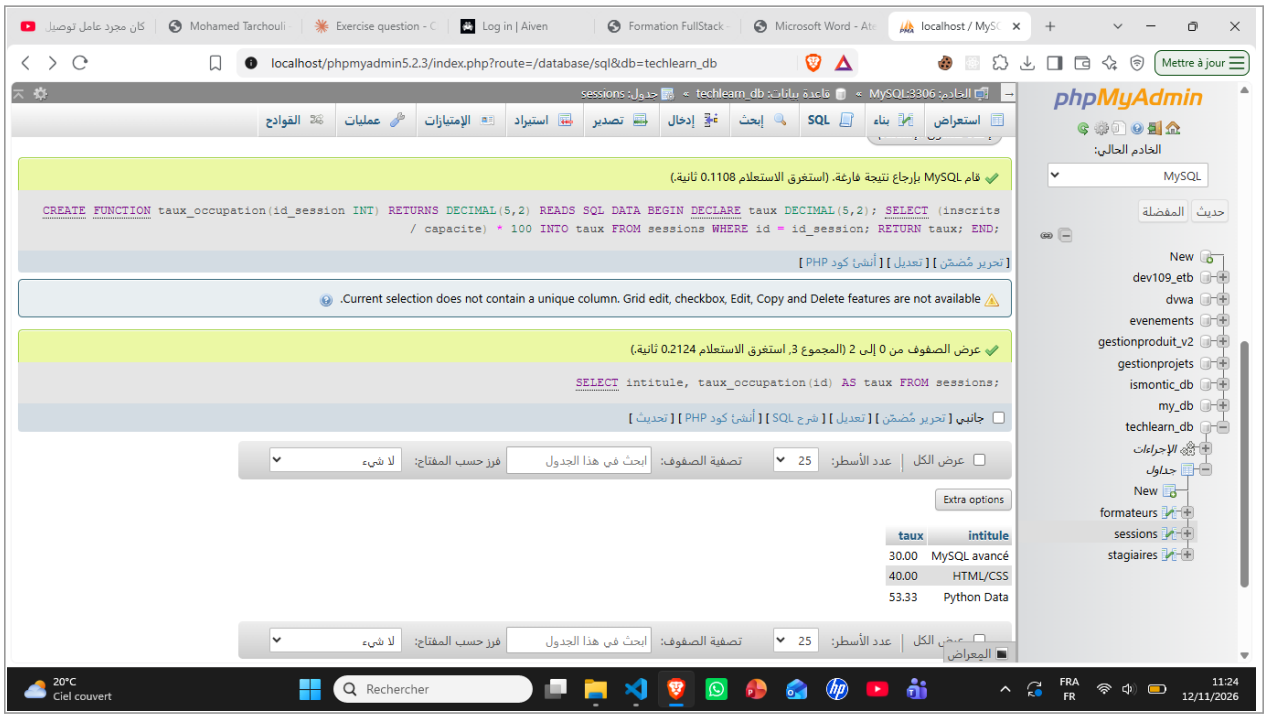


8. Exercice 4 – Fonction : taux d’occupation

Capture 18. Création de la fonction taux_occupation, qui calcule (inscrits / capacité) × 100. Le délimiteur $$ remplace /, car le signe / sert aussi à la division dans le calcul. La clause READS SQL DATA est ajoutée car la fonction lit la table. Elle est ensuite appelée sur toutes les sessions.

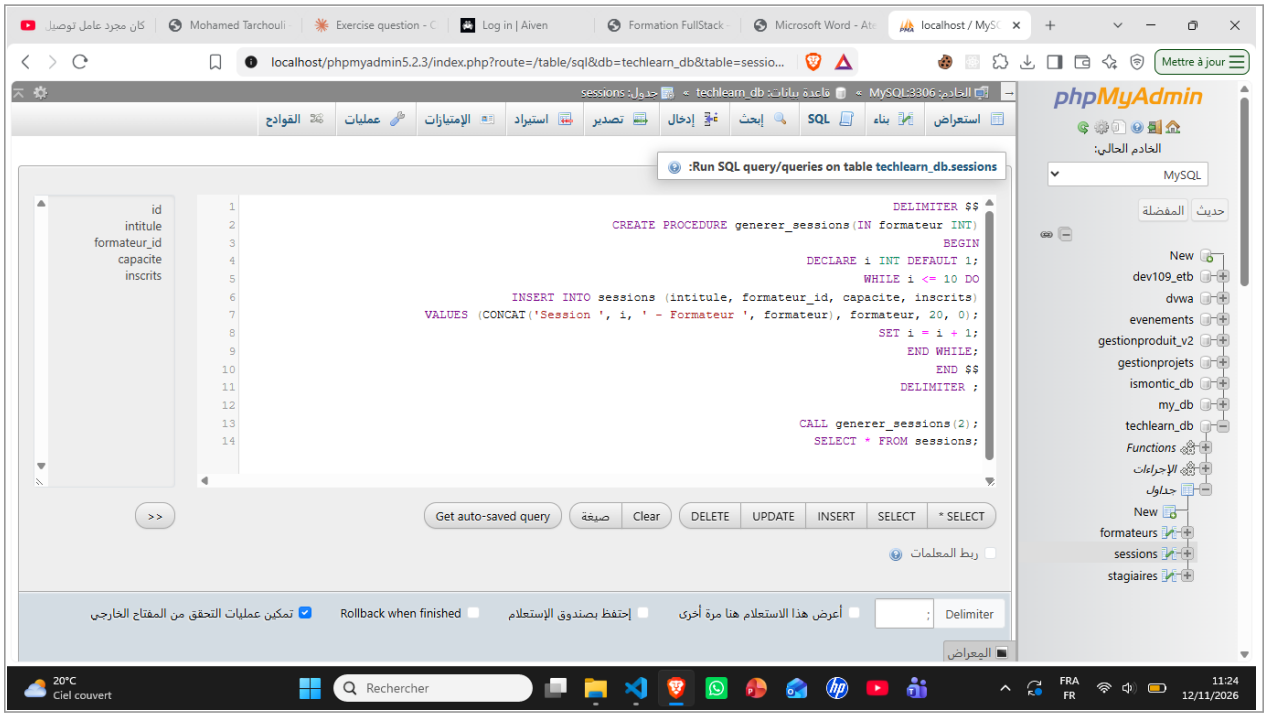


Capture 19. Résultat : MySQL avancé 30,00 % (6/20), HTML/CSS 40,00 % (10/25), Python Data 53,33 % (8/15).

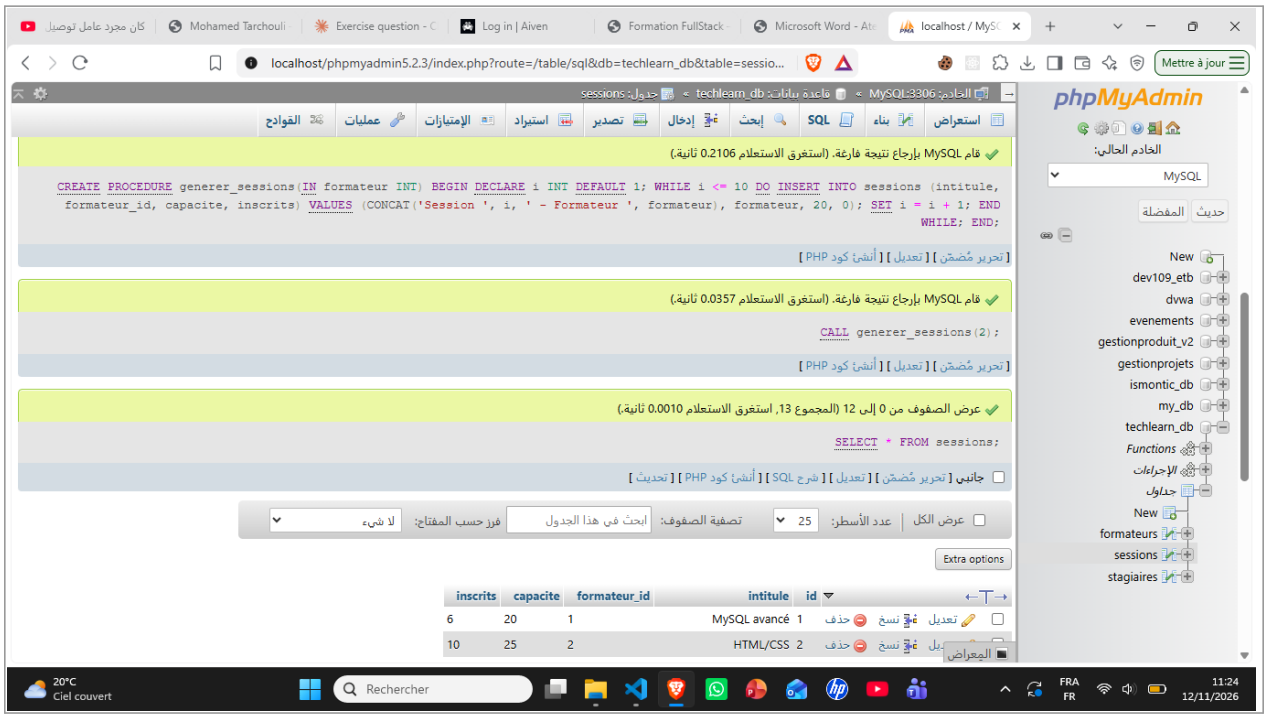


9. Exercice 5 – Boucle et automatisation

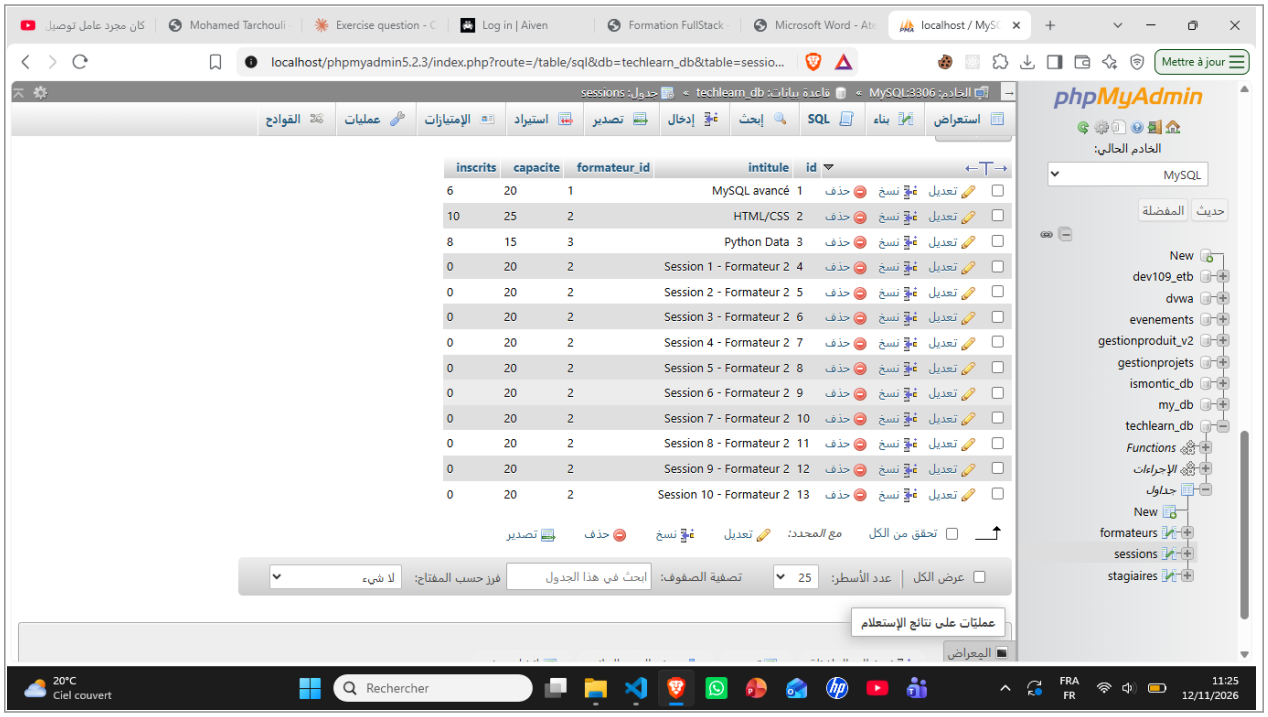
Capture 20. Saisie de la procédure generer_sessions : une boucle WHILE insère 10 sessions de capacité 20 et 0 inscrit pour le formateur passé en paramètre. Appel avec le formateur 2, suivi d'un SELECT sur sessions.



Capture 21. La procédure est créée, l’appel generer_sessions(2) réussit et le SELECT retourne 13 lignes (3 initiales + 10 générées).

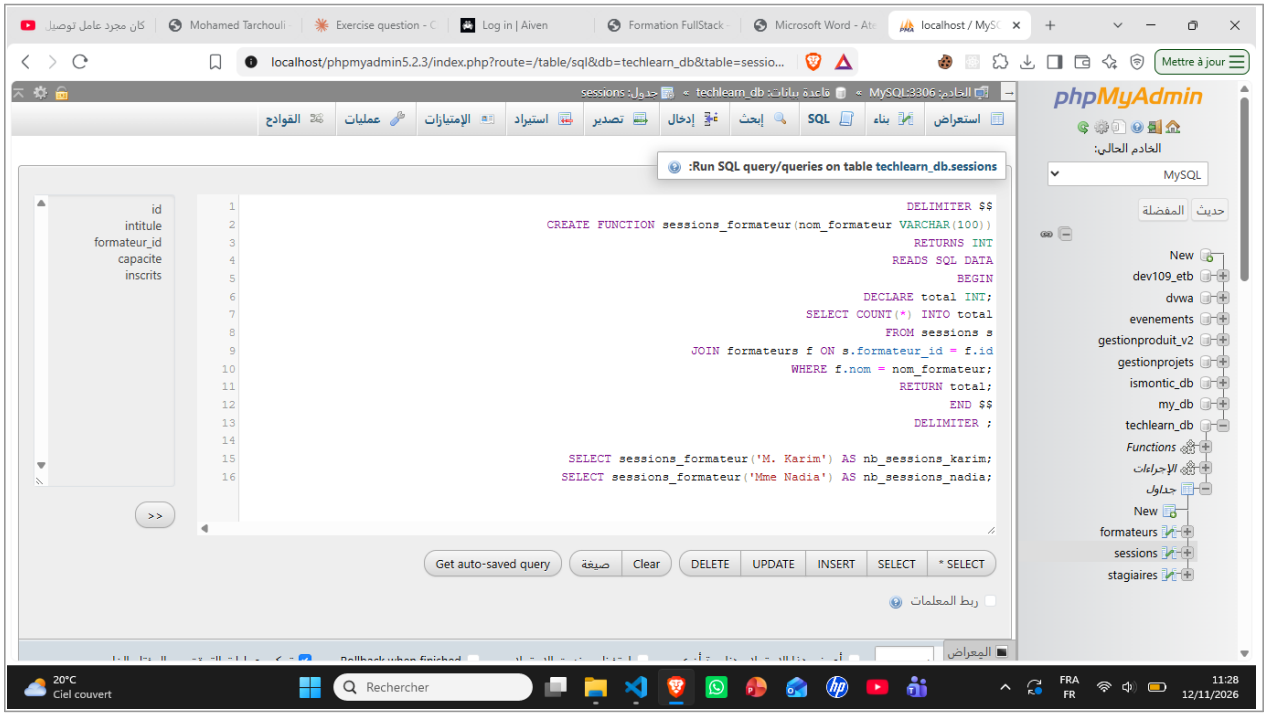


Capture 22. Contenu de la table sessions : les sessions 4 à 13 (« Session 1 – Formateur 2 » à « Session 10 – Formateur 2 ») ont été créées avec 0 inscrit.

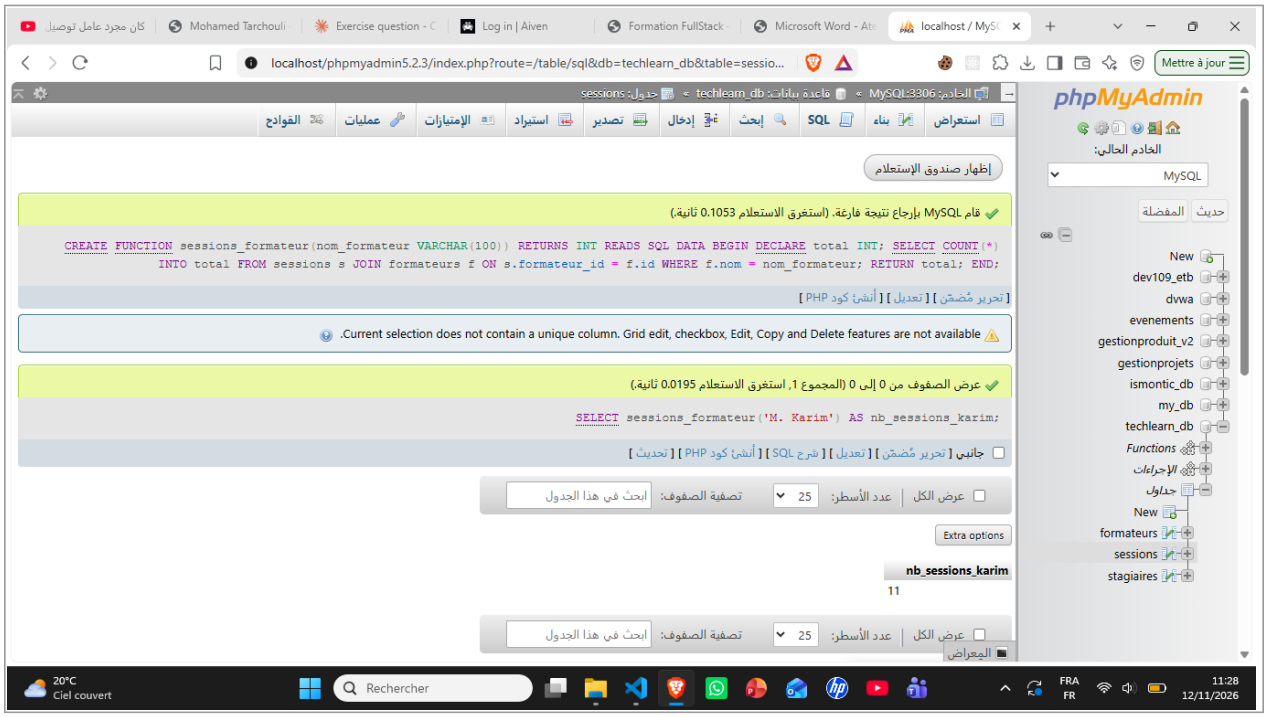


10. Défi 1 – Fonction sessions_formateur

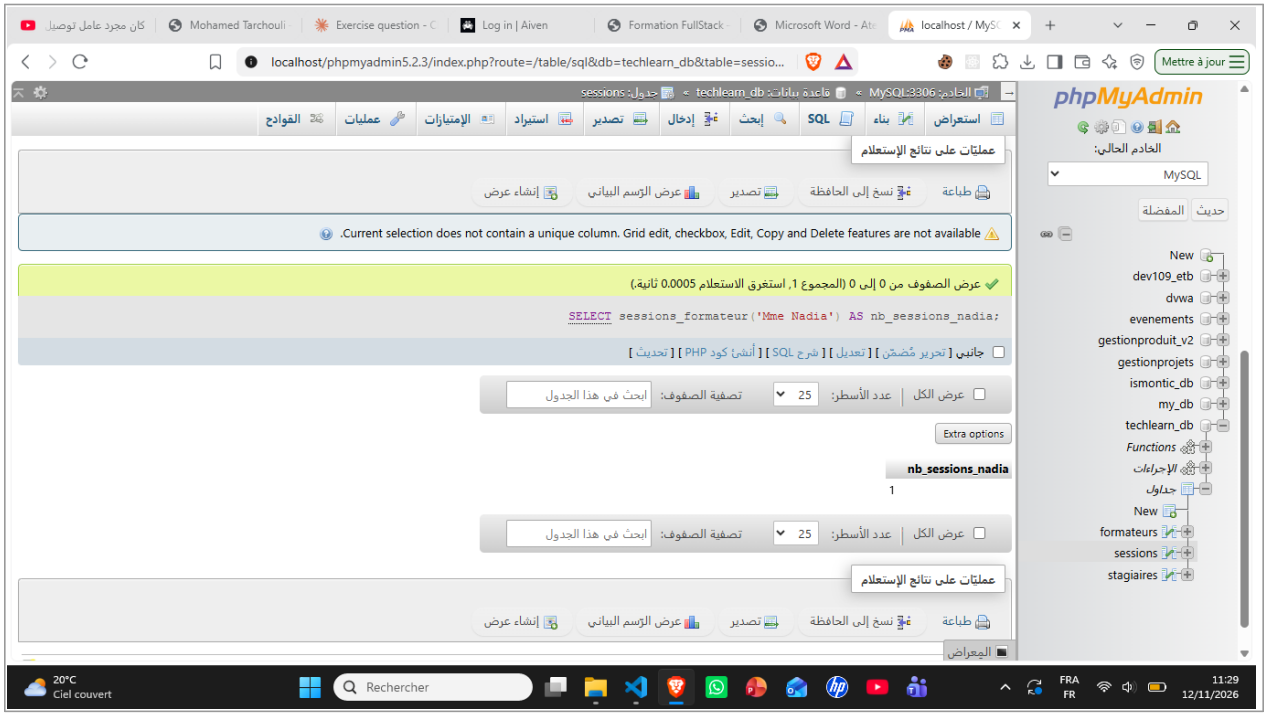
Capture 23. Création de la fonction sessions_formateur. Elle prend le nom d'un formateur, relie sessions et formateurs par une jointure (JOIN) et compte les sessions correspondantes.



Capture 24. Test avec « M. Karim » : résultat 11 (sa session HTML/CSS plus les 10 sessions générées).

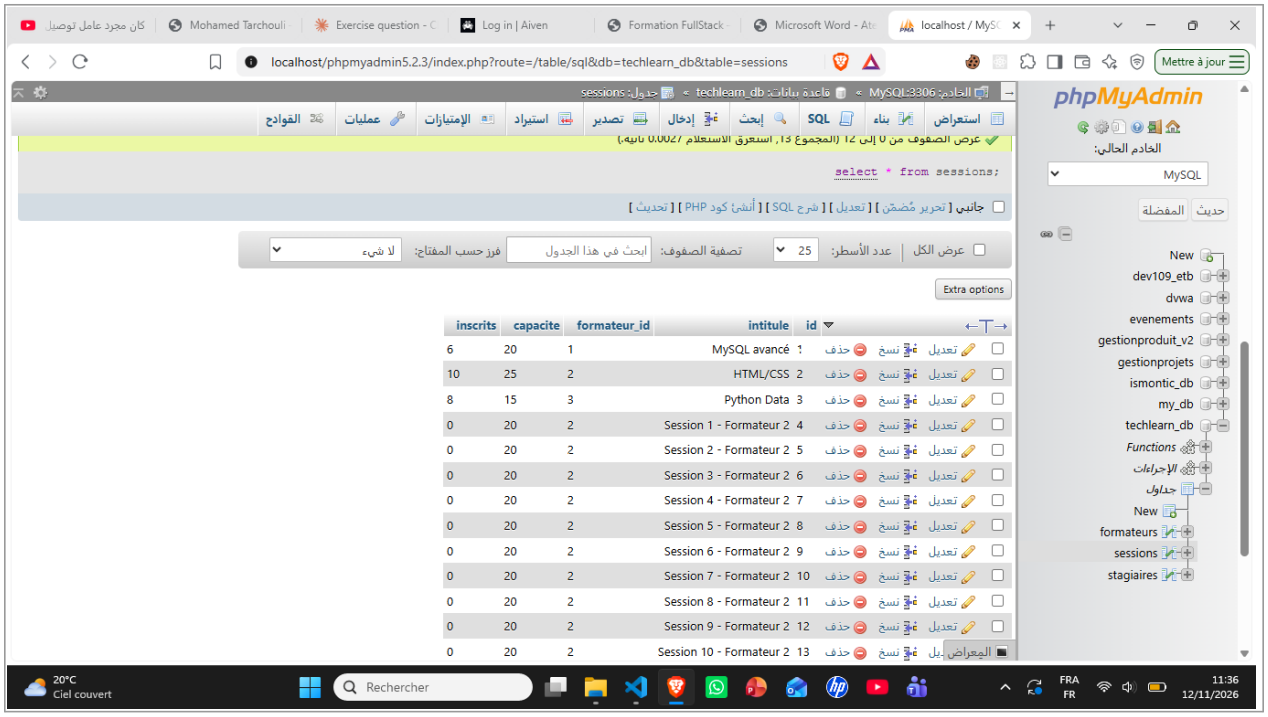


Capture 25. Test avec « Mme Nadia » : résultat 1 (la session MySQL avancé).



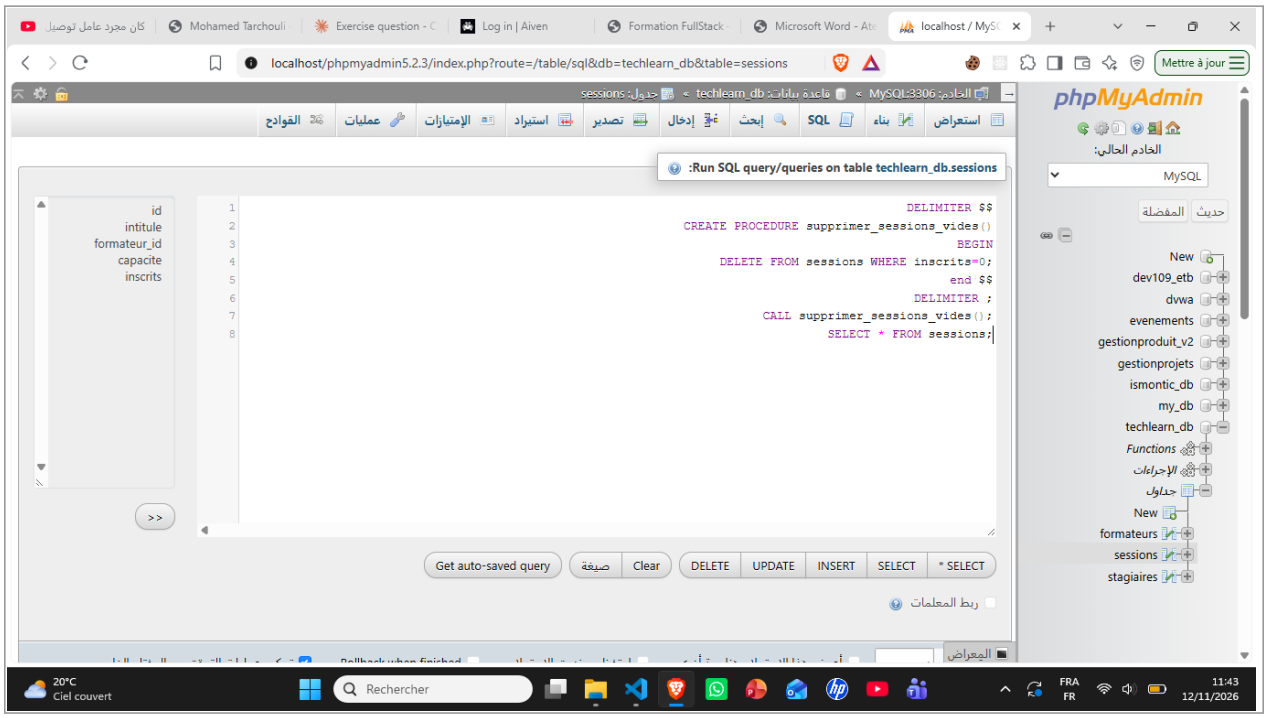
11. Défi 2 – Procédure supprimer_sessions_vides

Capture 26. État de la table sessions avant la suppression : 13 lignes, dont 10 sessions avec 0 inscrit.

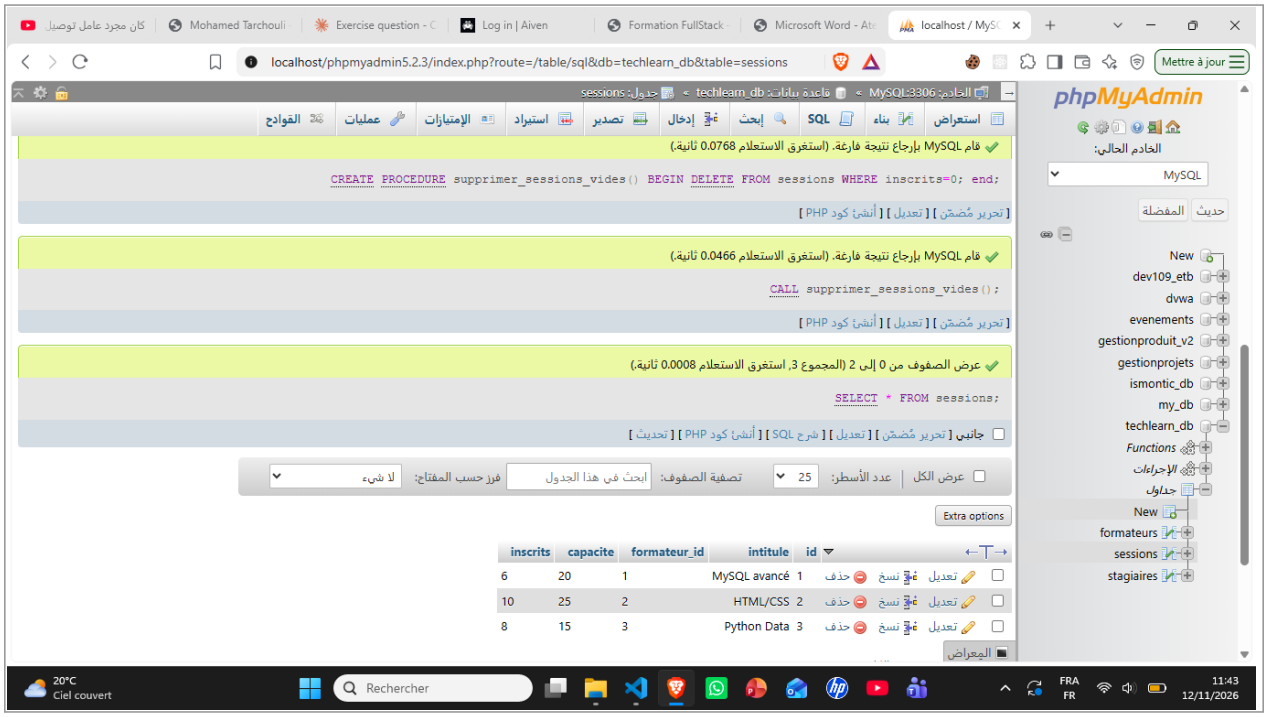


inscrits	capacite	formateur_id	intitule	id
6	20	1	MySQL avancé	1
10	25	2	HTML/CSS	2
8	15	3	Python Data	3
0	20	2	Session 1 - Formateur 2	4
0	20	2	Session 2 - Formateur 2	5
0	20	2	Session 3 - Formateur 2	6
0	20	2	Session 4 - Formateur 2	7
0	20	2	Session 5 - Formateur 2	8
0	20	2	Session 6 - Formateur 2	9
0	20	2	Session 7 - Formateur 2	10
0	20	2	Session 8 - Formateur 2	11
0	20	2	Session 9 - Formateur 2	12
0	20	2	Session 10 - Formateur 2	13

Capture 27. Création de la procédure supprimer_sessions_vides, qui exécute DELETE FROM sessions WHERE inscrits = 0, puis appel de la procédure et SELECT de contrôle.

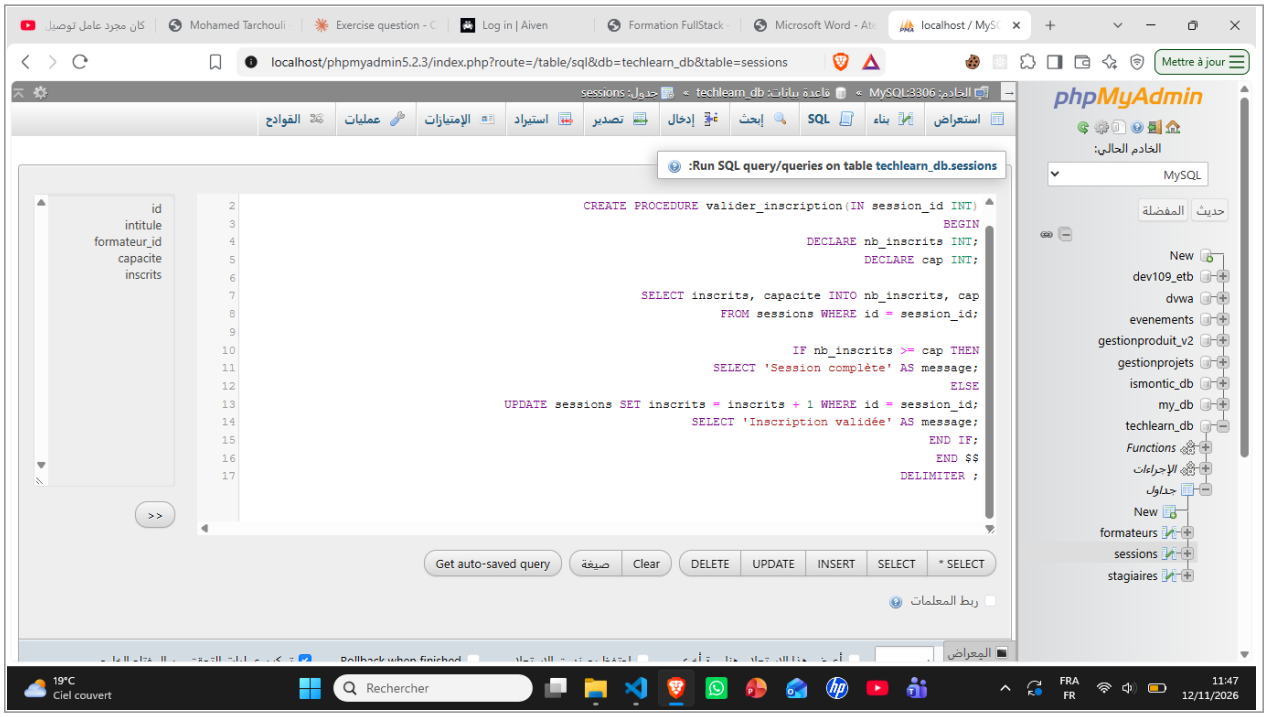


Capture 28. Résultat : la procédure est créée et exécutée. Il ne reste plus que les 3 sessions ayant des inscrits.

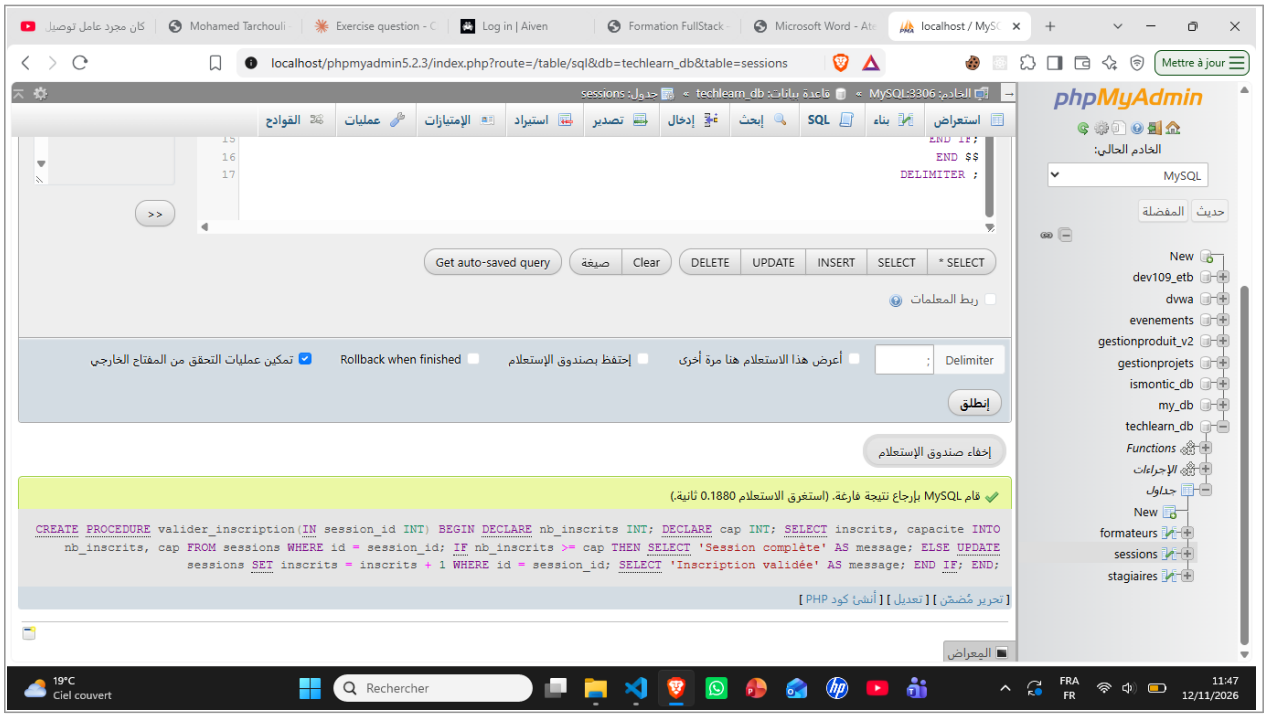


12. Défi 3 – Procédure valider_inscription

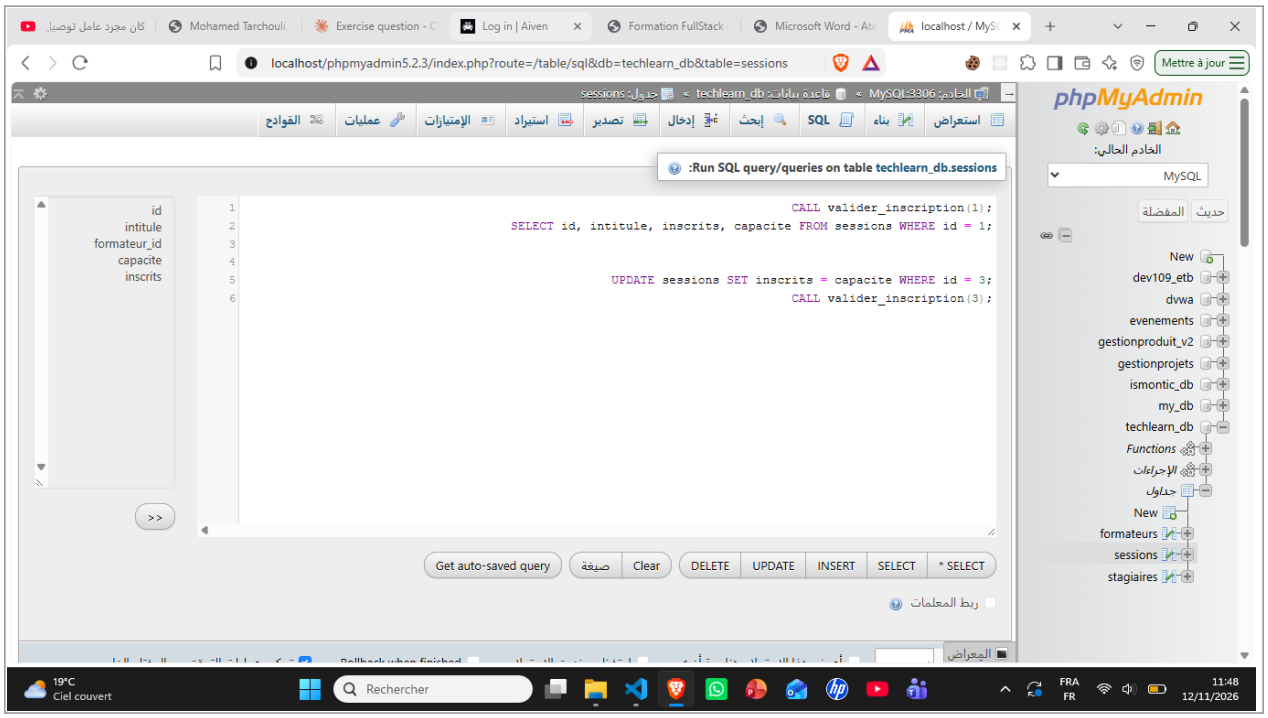
Capture 29. Code de la procédure valider_inscription. Elle lit les inscrits et la capacité de la session dans deux variables. Si inscrits ≥ capacité, elle affiche « Session complète ». Sinon, elle ajoute 1 inscrit et affiche « Inscription validée ».



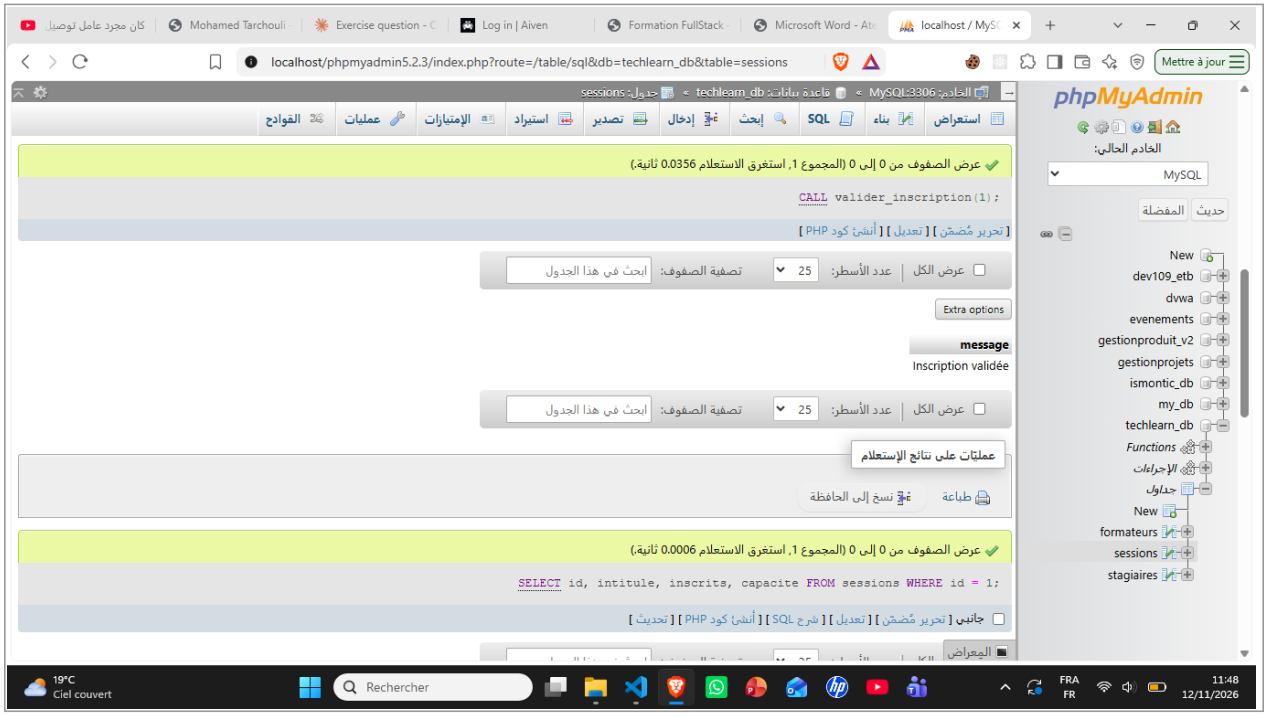
Capture 30. La procédure est créée avec succès.



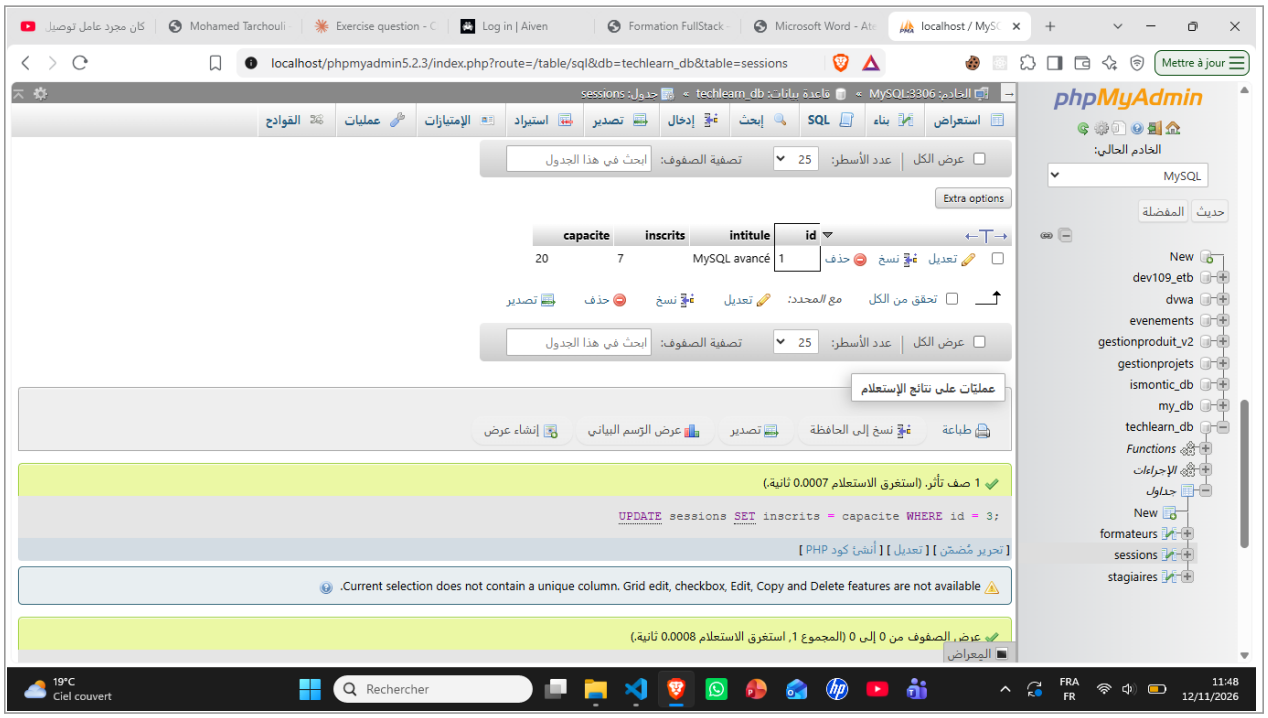
Capture 31. Scénarios de test : appel sur la session 1 (qui a encore de la place) avec un SELECT de contrôle, puis la session 3 est remplie à sa capacité (UPDATE) et testée à son tour.



Capture 32. Premier test (session 1) : le message « Inscription validée » est affiché.



Capture 33. Après le test, la session 1 passe à 7 inscrits sur 20. L'UPDATE qui remplit la session 3 a bien modifié 1 ligne.



Capture 34. Second test (session 3, pleine) : le message « Session complète » est affiché et aucune inscription n'est ajoutée.

